

Contents

Lösungen — NUR für den Dozenten	1
Epic 1	1
Epic 2	4
Epic 3	7
Epic 4	10
Epic 5	11

Lösungen — NUR für den Dozenten

Nicht an Schüler weitergeben, bevor sie es selbst versucht haben.

Alle Lösungen nutzen die Toolkit-Funktionen aus `framework.arena` (siehe [docs/toolkit-referenz.md](#)) — genau wie es die Schüler auch tun sollen. Bewusst **kein** kompaktes “kotliniges” `when/ Elvis`-Feuerwerk, sondern geradlinige `if/else`-Blöcke mit sprechenden Variablen: so sieht es aus, wenn Schüler es selbst erarbeiten, und das ist der realistische Maßstab beim Review.

Wofür nutzen? - Als Vergleichsmaßstab beim Review (“das ist völlig ok so, muss nicht kürzer”). - Als Hilfe für Teams, die nicht weiterkommen. - Um Schülern zu zeigen: **funktionierend und verständlich schlägt kurz und clever.**

Alle Beispiele gehen von diesen Imports aus:

```
import framework.arena.Action
import framework.arena.Direction
import framework.arena.RobotBrain
import framework.arena.Sensors
import framework.arena.approachDirectionTo
import framework.arena.canShoot
import framework.arena.directionTo
import framework.arena.directionToNearestEdge
import framework.arena.enemiesInLineOfSight
import framework.arena.fleeDirectionFrom
import framework.arena.isInsideArena
import framework.arena.isNearCenter
import framework.arena.manhattanDistanceTo
import framework.arena.nearestEnemy
import framework.arena.nearestEnemyInLineOfSight
import framework.arena.weakestEnemy
```

Epic 1

1.1 — Zufällige Bewegung

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val zufallsRichtung = Direction.entries.random()
        return Action.Move(zufallsRichtung)
    }
}
```

Hinweise für den Dozenten: - Bewusst eine eigene Variable `zufallsRichtung` statt alles in eine Zeile — für Anfänger leichter zu lesen und zu debuggen. - `Direction.values().random()` (statt

.entries) ist genauso richtig; nicht bemängeln.

1.2 — Am Rand bleiben / Zentrum meiden

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        // Stehe ich zu nah an der Mitte?
        if (sensors.isNearCenter()) {
            // Ja -> zum nächsten Rand laufen
            return Action.Move(sensors.directionToNearestEdge())
        }

        // Nein, weit genug weg -> einfach zufällig bewegen
        return Action.Move(Direction.entries.random())
    }
}
```

Hinweise für den Dozenten: - `sensors.isNearCenter()` und `sensors.directionToNearestEdge()` nehmen den Schülern die komplette Rand-/Mitte-Geometrie ab (Mittelpunkt berechnen, Abstand zu allen 4 Rändern vergleichen). Vorher musste das von Hand ausgerechnet werden — das war für Story 1.2 unverhältnismäßig viel Aufwand im Vergleich zum Lernziel (“auf Sensordaten reagieren”). - `isNearCenter()` hat einen optionalen `margin`-Parameter (Default 2) — für die Story reicht der Default.

1.3 — Flucht bei niedriger Gesundheit

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        // Gibt es überhaupt einen Gegner?
        val naechster = sensors.nearestEnemy() ?: return Action.Wait
        val meinePosition = sensors.self.position

        // Wenig HP -> WEG vom Gegner laufen
        if (sensors.self.health < 20) {
            val fluchtRichtung = meinePosition.fleeDirectionFrom(naechster.position)!!
            return Action.Move(fluchtRichtung)
        }

        // Genug HP -> ganz normal auf den Gegner ZU laufen
        val angriffsRichtung = meinePosition.approachDirectionTo(naechster.position)!!
        return Action.Move(angriffsRichtung)
    }
}
```

Hinweise für den Dozenten: - `sensors.nearestEnemy()` ersetzt die komplette Von-Hand-Suchschleife (nächsten Gegner per Manhattan-Distanz finden) — liefert `null`, wenn kein Gegner mehr lebt, daher der frühe `?: return Action.Wait`. - `fleeDirectionFrom/approachDirectionTo` liefern nur `null`, wenn beide Positionen identisch sind (praktisch nie der Fall, da zwei Bots nicht dasselbe Feld belegen können) — `!!` ist hier vertretbar. - Kritische Review-Stelle: läuft der Bot bei `health < 20` wirklich **weg**? `fleeDirectionFrom` übernimmt das Vorzeichen-Denken jetzt selbst, aber Schüler sollten trotzdem verstehen, *warum* `flee` die Umkehrung von `approach` ist.

1.4 — Keine Bewegung verschwenden (Randcheck)

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val meinePosition = sensors.self.position
        val richtung = Direction.entries.random()
        val zielPosition = meinePosition.moved(richtung)

        // Würde das die Arena verlassen? Dann eine andere Richtung nehmen
        if (!zielPosition.isInsideArena(sensors.arenaWidth, sensors.arenaHeight)) {
            val gueltigeRichtung = Direction.entries.first {
                meinePosition.moved(it).isInsideArena(sensors.arenaWidth, sensors.arenaHeight)
            }
            return Action.Move(gueltigeRichtung)
        }

        return Action.Move(richtung)
    }
}
```

Hinweise für den Dozenten: - `Position.moved(direction)` (aus `Models.kt`, nicht aus dem Toolkit) liefert die Zielposition, `isInsideArena` prüft sie gegen die Arena-Grenzen — beide zusammen ersetzen den manuellen Vergleich mit `0/arenaWidth - 1`. - `Direction.entries.first { ... }` findet die erste gültige Richtung; es gibt auf einem 10×10-Feld nie eine Situation, in der **keine** der vier Richtungen gültig ist (Ecken haben immer mindestens zwei gültige Richtungen). - Diese Story ist bewusst als Vorstufe zu 1.6 gedacht (Wandvermeidung beim Fliehen) — hier erst mal ohne HP-Bezug, nur der reine Randcheck.

1.5 — Sicherheitsabstand zum nächsten Gegner halten

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val naechster = sensors.nearestEnemy() ?: return Action.Wait
        val meinePosition = sensors.self.position

        val abstand = meinePosition.manhattanDistanceTo(naechster.position)

        // Zu nah dran -> Abstand vergrößern
        if (abstand < 2) {
            val fluchtRichtung = meinePosition.fleeDirectionFrom(naechster.position)!!
            return Action.Move(fluchtRichtung)
        }

        // Genug Abstand -> normal weiter (hier: Zufallsbewegung aus 1.1)
        return Action.Move(Direction.entries.random())
    }
}
```

Hinweise für den Dozenten: - `manhattanDistanceTo` ersetzt die manuelle `abs(dx) + abs(dy)`-Rechnung. - Der Schwellenwert 2 ist willkürlich (Story verlangt nur “selbst gewählt”) — jede sinnvolle Zahl akzeptieren, solange sie im Code als klarer Vergleich erkennbar ist. - Unterschied zu 1.3: hier

geht es nur um **Distanz**, nicht um eigene HP — bewusst als eigenständiges Kriterium, nicht als Ersatz für die Flucht-Story.

1.6 — Fluchtrichtung mit Wandvermeidung kombinieren

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val naechster = sensors.nearestEnemy() ?: return Action.Wait
        val meinePosition = sensors.self.position

        if (sensors.self.health < 20) {
            val fluchtRichtung = meinePosition.fleeDirectionFrom(naechster.position)!!
            val fluchtZiel = meinePosition.moved(fluchtRichtung)

            // Würde die Flucht gegen die Wand laufen? Andere Richtung suchen
            if (!fluchtZiel.isInsideArena(sensors.arenaWidth, sensors.arenaHeight)) {
                val ausweichRichtung = Direction.entries.first {
                    meinePosition.moved(it).isInsideArena(sensors.arenaWidth, sensors.arenaHeight)
                }
                return Action.Move(ausweichRichtung)
            }

            return Action.Move(fluchtRichtung)
        }

        val angriffsRichtung = meinePosition.approachDirectionTo(naechster.position)!!
        return Action.Move(angriffsRichtung)
    }
}
```

Hinweise für den Dozenten: - Direkte Erweiterung von 1.3 und 1.4 — guter Moment, um Schülern zu zeigen, dass frühere Storys wiederverwendet statt neu erfunden werden. - Wichtige Review-Stelle: die Ausweich-Richtung muss **nicht** zwingend “noch weiter weg vom Gegner” sein — für diese Story reicht “irgendeine gültige Richtung”, das ist bewusst einfach gehalten.

Epic 2

2.1 — Dauerfeuer in feste Richtung

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        return Action.Shoot(Direction.EAST)
    }
}
```

Nichts zu erklären — gegen StillstandBot testen lassen, damit Treffer sichtbar werden.

2.2 — Auf Gegner in Sichtlinie zielen

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
```

```

// Nächsten Gegner in Sichtlinie suchen
val ziel = sensors.nearestEnemyInLineOfSight() ?: return Action.Wait

val schussRichtung = sensors.self.position.directionTo(ziel.position)!!
return Action.Shoot(schussRichtung)
}
}

```

Hinweise für den Dozenten: - `sensors.nearestEnemyInLineOfSight()` übernimmt beide Schritte auf einmal: erst nach Ausrichtung filtern (gleiche Reihe/Spalte), dann den nächsten davon wählen. Vorher zwei Schritte von Hand (Schleife + Filter). - `directionTo` liefert nur dann eine Richtung, wenn die Ziel-Position wirklich ausgerichtet ist — genau das ist hier durch `nearestEnemyInLineOfSight()` bereits sichergestellt, daher ist `!!` sicher. - Wichtige Review-Stelle bleibt bestehen: Schüler sollten grob verstehen, dass Y nach unten wächst — auch wenn sie die Formel nicht mehr selbst schreiben.

2.3 — Gegner verfolgen, wenn nicht in Schusslinie

```

class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val naechster = sensors.nearestEnemy() ?: return Action.Wait
        val meinePosition = sensors.self.position

        if (sensors.canShoot(naechster)) {
            val schussRichtung = meinePosition.directionTo(naechster.position)!!
            return Action.Shoot(schussRichtung)
        } else {
            val bewegungsRichtung = meinePosition.approachDirectionTo(naechster.position)
            return Action.Move(bewegungsRichtung)
        }
    }
}

```

Hinweise für den Dozenten: - Das ist im Kern der ChaserBot, nur mit `if/else` statt `when`. - `sensors.canShoot(naechster)` prüft die Ausrichtung, ohne dass Schüler `x == x || y == y` selbst hinschreiben müssen. - `approachDirectionTo` wählt selbst die Achse mit dem größeren Abstand zuerst — die “diagonale Annäherung über mehrere Ticks” passiert also automatisch.

2.4 — Nicht ins Leere schießen

```

class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val naechster = sensors.nearestEnemy() ?: return Action.Wait
        val meinePosition = sensors.self.position

        // Nur schießen, wenn wirklich getroffen werden kann
        if (sensors.canShoot(naechster)) {
            val schussRichtung = meinePosition.directionTo(naechster.position)!!
            return Action.Shoot(schussRichtung)
        }

        // Sonst lieber bewegen statt sinnlos zu schießen
    }
}

```

```

        return Action.Move(Direction.entries.random())
    }
}

```

Hinweise für den Dozenten: - Reine Vorstufe zu 2.2/2.3 — hier ist das Bewegen im else-Zweig noch nicht zielgerichtet (`entries.random()`), das kommt erst mit 2.3. - Lernpunkt: **erst prüfen, dann schießen** statt einfach drauflos in eine feste Richtung wie in 2.1.

2.5 — Schwächsten Gegner in Sichtlinie zuerst angreifen

```

class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val gegnerInLinie = sensors.enemiesInLineOfSight()

        // Unter den treffbaren Gegnern den schwächsten wählen
        val ziel = gegnerInLinie.minByOrNull { it.health }

        if (ziel != null) {
            val schussRichtung = sensors.self.position.directionTo(ziel.position)!!
            return Action.Shoot(schussRichtung)
        }

        // Niemand in Sichtlinie -> verfolgen wie in 2.3
        val naechster = sensors.nearestEnemy() ?: return Action.Wait
        val bewegungsRichtung = sensors.self.position.approachDirectionTo(naechster.position)
        return Action.Move(bewegungsRichtung)
    }
}

```

Hinweise für den Dozenten: - `sensors.enemiesInLineOfSight()` liefert **alle** treffbaren Gegner (nicht nur den nächsten) — genau das braucht diese Story, um darunter nach HP zu wählen. Bewusst `minByOrNull { it.health }` statt `weakestEnemy()`, weil `weakestEnemy()` über **alle** Gegner geht, nicht nur die in Sichtlinie. - Diese Story ist eine leichtere Vorstufe zu 3.2 (dort wird die Zielpriorisierung mit `weakestEnemy()` global statt nur unter den sichtbaren Gegnern gemacht) — nicht verwechseln.

2.6 — Rückzug beim Schießen vermeiden

```

class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val naechster = sensors.nearestEnemy() ?: return Action.Wait
        val meinePosition = sensors.self.position

        if (sensors.canShoot(naechster)) {
            val abstand = meinePosition.manhattanDistanceTo(naechster.position)

            // Zu nah dran, obwohl ich schon schießen könnte -> zurückweichen
            if (abstand < 2) {
                val fluchtRichtung = meinePosition.fleeDirectionFrom(naechster.position)
                return Action.Move(fluchtRichtung)
            }
        }
    }
}

```

```

        val schussRichtung = meinePosition.directionTo(naechster.position)!!
        return Action.Shoot(schussRichtung)
    }

    val bewegungsRichtung = meinePosition.approachDirectionTo(naechster.position)!!
    return Action.Move(bewegungsRichtung)
}
}

```

Hinweise für den Dozenten: - Erweitert 2.3 um einen zusätzlichen Fall zwischen “schießen” und “verfolgen” — Reihenfolge der Prüfung ist hier wichtig: erst canShoot, dann innerhalb davon den Abstand prüfen. - manhattanDistanceTo wie schon in 1.5 — guter Moment, um die Wiederverwendung der Toolkit-Funktion über beide Epics hinweg zu zeigen.

Epic 3

3.1 — Zustandsmaschine (Patrouille / Angriff / Flucht)

```

class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val meinePosition = sensors.self.position

        // 1. Gibt es einen Gegner? Wenn nein -> PATROUILLE
        val naechster = sensors.nearestEnemy()
        if (naechster == null) {
            return Action.Move(Direction.entries.random())
        }

        // 2. Wenig HP? -> FLUCHT (hat Vorrang vor Angriff!)
        if (sensors.self.health < 20) {
            val fluchtRichtung = meinePosition.fleeDirectionFrom(naechster.position)!!
            return Action.Move(fluchtRichtung)
        }

        // 3. Sonst -> ANGRIFF (in Linie schießen, sonst hinlaufen)
        if (sensors.canShoot(naechster)) {
            val schussRichtung = meinePosition.directionTo(naechster.position)!!
            return Action.Shoot(schussRichtung)
        } else {
            val bewegungsRichtung = meinePosition.approachDirectionTo(naechster.position)!!
            return Action.Move(bewegungsRichtung)
        }
    }
}

```

Hinweise für den Dozenten: - Hier bewusst **ohne** enum class BotState. Die drei Zustände sind stattdessen als klar kommentierte Blöcke (*// 1. ...*, *// 2. ...*, *// 3. ...*) umgesetzt. Die Story verlangt “mindestens 3 erkennbare Zustände” — kommentierte if-Blöcke erfüllen das. - **Zentrale Review-Stelle:** Die Reihenfolge muss stimmen — erst PATROUILLE (kein Gegner), dann FLUCHT (wenig HP), dann ANGRIFF. Wird Angriff vor Flucht geprüft, stirbt der fast tote Bot beim Angreifen. - Die Bewegungs-/Schuss-Blöcke sind aus 1.3 und 2.3 übernommen. Genau das ist der Lernpunkt: nichts Neues, nur **sortiert**.

3.2 — Zielpriorisierung (schwächster Gegner zuerst)

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        // ZIEL-REGEL: den Gegner mit den wenigsten HP zuerst angreifen
        val ziel = sensors.weakestEnemy() ?: return Action.Wait
        val meinePosition = sensors.self.position

        // Ab hier genau wie in 2.3, nur mit `ziel` statt `naechster`
        if (sensors.canShoot(ziel)) {
            val schussRichtung = meinePosition.directionTo(ziel.position)!!
            return Action.Shoot(schussRichtung)
        } else {
            val bewegungsRichtung = meinePosition.approachDirectionTo(ziel.position)!!
            return Action.Move(bewegungsRichtung)
        }
    }
}
```

Hinweise für den Dozenten: - Einziger echter Unterschied zu 2.3: `sensors.weakestEnemy()` statt `sensors.nearestEnemy()` — genau der Lernpunkt der Story (Kriterium tauschen, Struktur bleibt).
- Anderes sinnvolles Kriterium (z.B. wieder nächster Gegner) ebenfalls akzeptieren, solange es klar benannt/kommentiert ist.

3.3 — Kür-Aufgabe

Keine feste Lösung (offen). Als Beispiel eine typische, gut machbare Schüler-Idee: **“Nur schießen, wenn genau EIN Gegner in Linie steht”** (sonst lieber ausweichen, um nicht selbst getroffen zu werden):

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val gegnerInLinie = sensors.enemiesInLineOfSight()

        // Nur schießen, wenn genau einer in Linie steht (sicherer Treffer)
        if (gegnerInLinie.size == 1) {
            val ziel = gegnerInLinie[0]
            val schussRichtung = sensors.self.position.directionTo(ziel.position)!!
            return Action.Shoot(schussRichtung)
        }

        // Sonst: einfach ausweichen / bewegen
        return Action.Move(Direction.entries.random())
    }
}
```

Hinweis für den Dozenten: Die Idee muss mit `Sensors/Action` ausdrückbar sein (nur aktuelle Momentaufnahme, keine Zukunft, keine Historie ohne `var`). Vor dem Start kurz Machbarkeit bestätigen. `sensors.enemiesInLineOfSight()` (zusätzlicher Import) nimmt hier das Filtern ab.

3.4 — Zeitgesteuerte Startphase (Patrouille zuerst)

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {  
  
    companion object {  
        const val PATROUILLE_TICKS = 10  
    }  
  
    override fun decide(sensors: Sensors): Action {  
        // In den ersten Ticks nur patrouillieren, egal was sensors.others zeigt  
        if (sensors.tick < PATROUILLE_TICKS) {  
            return Action.Move(Direction.entries.random())  
        }  
  
        val naechster = sensors.nearestEnemy() ?: return Action.Wait  
        val meinePosition = sensors.self.position  
  
        if (sensors.canShoot(naechster)) {  
            val schussRichtung = meinePosition.directionTo(naechster.position)!!  
            return Action.Shoot(schussRichtung)  
        }  
  
        val bewegungsRichtung = meinePosition.approachDirectionTo(naechster.position)!!  
        return Action.Move(bewegungsRichtung)  
    }  
}
```

Hinweise für den Dozenten: - `sensors.tick` wurde bisher in keiner anderen Lösung gebraucht — kurz zeigen, dass `Sensors` das mitliefert (siehe `Models.kt`). - Die Konstante als `companion object`-Wert ist eine Möglichkeit, reicht aber auch als einfache `val` außerhalb der Klasse — Hauptsache klar benannt. - Guter Anknüpfungspunkt für Diskussion: was, wenn ein Gegner in den ersten Ticks direkt neben dem Bot steht? Für diese Story bewusst nicht behandelt (Story verlangt nur reines Zeit-Gate, keine Ausnahme für Notfälle).

3.5 — Abklingzeit nach der Flucht

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {  
  
    companion object {  
        const val ABKLINGZEIT_TICKS = 5  
    }  
  
    var ticksSeitFlucht = ABKLINGZEIT_TICKS  
  
    override fun decide(sensors: Sensors): Action {  
        val fliehtGerade = sensors.self.health < 20  
  
        if (fliehtGerade) {  
            ticksSeitFlucht = 0  
        } else {  
            ticksSeitFlucht++  
        }  
  
        val nochVorsichtig = ticksSeitFlucht < ABKLINGZEIT_TICKS
```

```

    if (fliehtGerade || nochVorsichtig) {
        val naechster = sensors.nearestEnemy()
        if (naechster == null) {
            return Action.Move(sensors.directionToNearestEdge())
        }
        val fluchtRichtung = sensors.self.position.fleeDirectionFrom(naechster.position)
        return Action.Move(fluchtRichtung)
    }

    val naechster = sensors.nearestEnemy() ?: return Action.Wait
    val meinePosition = sensors.self.position
    if (sensors.canShoot(naechster)) {
        val schussRichtung = meinePosition.directionTo(naechster.position)!!
        return Action.Shoot(schussRichtung)
    }
    val bewegungsRichtung = meinePosition.approachDirectionTo(naechster.position)!!
    return Action.Move(bewegungsRichtung)
}
}

```

Hinweise für den Dozenten: - Erste Lösung, die einen echten Zähler über mehrere Ticks hinweg braucht — var ticksSeitFlucht als Klassen-Property, startet absichtlich auf ABKLINGZEIT_TICKS (nicht 0), damit der Bot zu Matchbeginn nicht direkt im “vorsichtig”-Zustand hängt. - sensors.directionToNearestEdge() als Fallback, falls kein Gegner mehr da ist, aber der Bot noch in der Abklingzeit steckt — sonst müsste man sonst wieder eine Ausnahme für “kein Gegner” separat behandeln. - Guter Moment, um mit Schülern über den Unterschied zwischen “bei jedem decide()-Aufruf neu berechnet” (wie in 3.1) und “über Ticks gemerkt” (wie hier) zu sprechen.

Epic 4

4.1 — Unit-Test

```
package bots.teama
```

```

import framework.arena.Action
import framework.arena.Direction
import framework.arena.Position
import framework.arena.RobotState
import framework.arena.Sensors
import kotlin.test.Test
import kotlin.test.assertEquals

```

```

class MeinBotTest {
    @Test
    fun `schießt nach Osten wenn Gegner rechts in gleicher Zeile steht`() {
        // Ich stehe bei (2,5), Gegner bei (7,5) -> gleiche Zeile, rechts von mir
        val ich = RobotState(id = "bot-0", teamName = "Team A", position = Position(2, 5))
        val gegner = RobotState(id = "bot-1", teamName = "Team B", position = Position(7, 5))
        val sensors = Sensors(
            self = ich,
            others = listOf(gegner),
            arenaWidth = 10,

```

```

        arenaHeight = 10,
        tick = 1
    )

    val aktion = MeinBot().decide(sensors)

    assertEquals(Action.Shoot(Direction.EAST), aktion)
}

```

Hinweise für den Dozenten: - Setzt voraus, dass MeinBot auf einen Gegner in gleicher Zeile mit Shoot(EAST) reagiert (z.B. die 2.2- oder 2.3-Lösung). Bei reinen Bewegungs- Bots muss der erwartete Wert entsprechend angepasst werden. - Datei muss unter `src/test/kotlin/bots/teamA/` liegen und `package bots.teamA` haben, sonst findet Gradle den Test nicht. - `./gradlew test` als Abnahme.

4.2 — Testduell protokollieren

Kein Code — organisatorisch.

Epic 5

Beide Storys organisatorisch, kein Code. Bei 5.1 prüfen, dass teamXBots nur die final gewollte Version enthält, und `./gradlew build` grün ist.